[UORA · VOICE-OVER]

8-Minute Narration

Full spoken script · ready for AI voice generation

This document is the complete word-for-word narration for the UORA demo video, timed to run approximately eight minutes at a natural ~150 words per minute pace. It is written to be read aloud verbatim — paste each segment (or the whole script) into an AI voice generator such as the one in Emergent, ElevenLabs, or any text-to-speech tool, and record the screen actions noted in amber to match.

TOTAL WORDS	~1,200 (≈ 8 minutes at 150 wpm)
SEGMENTS	8 — record continuously or one at a time
VOICE STYLE	Calm, confident, documentary. Not salesy.
PACE	Unhurried. Pause at every full stop.
PRONUNCIATION	UORA = "oo-OR-ah". gVisor = "gee-VY-zor".
PLAIN TEXT	UORA-Voiceover-8min.txt (paste-ready, no markup)

Segment 0 — Introduction

0:00 – 0:30

> ***On screen —*** *Title card: UORA logo, team name, participant card. Hold for 30 seconds.*

Hello, and welcome. My name is Vansh Gupta. I am a second-year Information Technology student at the Indian Institute of Information Technology, Bhopal — I I I T Bhopal. I am competing in the I I C P C twenty twenty-six hackathon as a solo participant under the team name UORA. Over the next eight minutes, I will walk you through what UORA is, how it is built, and a live demonstration running on Google Cloud. Everything you see is real — real benchmarks, real containers, real telemetry. Let us begin.

Segment 1 — The Hook

0:30 – 1:20

> ***On screen —*** *Landing page hero, animated latency stream pulsing on the right.*

Every electronic exchange on the planet runs on a single piece of software called a matching engine. It is the component that takes every buy order and every sell order, and decides — in a fraction of a millisecond — who trades with whom, at what price, and in what order. When it is fast and correct, markets are fair and liquid. When it is even slightly wrong — a fill out of order, a microsecond of unexpected delay — someone loses real money, and a regulator starts asking questions. And yet, the way most engineering teams test their matching engine before it goes live is shockingly informal. A few hand-written load scripts. A spreadsheet of latencies. A visual glance at an order book. There has never been a single platform that proves an engine is both fast and correct, under real pressure, with results anyone can reproduce. That is the problem I set out to solve. This is UORA.

Segment 2 — What UORA Is

■ 1:20 – 2:15

> ***On screen —*** *Scroll the landing page: hero stats, six-stage pipeline, feature cards.*

UORA stands for Unified Orderbook Resilience Architecture. In one sentence, it is a distributed benchmarking platform that takes a contestant's matching engine, runs it under fire, and ranks it on a live leaderboard. Here is how it works. You upload your engine as source code — C plus plus, Rust, Go, or Python. UORA compiles it inside a hardened, isolated sandbox. It then unleashes a distributed fleet of trading bots that replay real, deterministic market data from the LOBSTER dataset — the same nanosecond-resolution order flow a real exchange would see. While your engine is under load, UORA streams every latency measurement, every fill, and every correctness check, in real time, to a ranked dashboard. And critically — every number on that leaderboard is reproducible. Same market tape, same validator, same scoring formula, every single run. No synthetic results. No staged demos. Just real engines, proven under real conditions.

Segment 3 — The Architecture

■ 2:15 – 3:20

> ***On screen —*** *Architecture section: the four coloured layer pills; hover the components.*

Under the hood, UORA is built as four independent layers, and the key design decision is that they are completely decoupled. The first layer handles submission and sandboxing — authentication, file upload, object storage, and the secure build. The second layer is the benchmark and validation engine — the asynchronous bot fleet, the reference order book, and the four-level correctness validator. The third layer is telemetry and scoring — nanosecond timing captured at the proxy edge, stored in a time-series database, and fed into the composite score and the machine-learning anomaly detector. The fourth layer is the real-time leaderboard and user interface. What makes this architecture resilient is that these layers never share memory. They communicate only through Redis streams and publish-subscribe channels. That means any layer can be scaled up, restarted, or replayed completely independently of the others. A surge of submissions? Add more builders. A slow benchmark host? Replace it mid-flight. Nothing else even notices.

Segment 4 — Security & Sandboxing

■ 3:20 – 4:15

On screen — *Security stack diagram; the nested isolation layers.*

Now, there is an obvious danger here. We are taking untrusted code, written by someone we have never met, and running it on our infrastructure at full load. So security is not an afterthought — it is the foundation. Every submission runs behind five layers of progressively tighter isolation. At the outer edge, a gVisor user-space kernel intercepts and filters every single system call the engine attempts — direct access to the host kernel is simply impossible. Inside that, a seccomp-bpf profile enforces a deny-by-default policy: of over thirteen hundred Linux system calls, only the three hundred and twelve an engine legitimately needs are allowed; the rest are blocked at the kernel level. Network egress is set to none — the engine can only accept inbound orders from our bot fleet. CPU and memory are capped, and the container image is built from scratch, with no shell and no tools an attacker could use. This is the same class of isolation that the largest cloud providers use to run untrusted workloads.

Segment 5 — Validation: L1 to L4

■ 4:15 – 5:20

On screen — *Validation funnel diagram L1→L4; then the dashboard Validation tab.*

Speed without correctness is worthless. An engine that is blazingly fast but fills orders in the wrong sequence is not a fast engine — it is a broken one. So UORA validates every scored submission against a canonical reference order book, using four escalating levels of scrutiny. Level one checks price-time priority — that every fill respects the strict first-in, first-out ordering at each price. Level two checks the order lifecycle — that every order moves through valid states, from pending, to partially filled, to filled or cancelled. Level three checks market invariants — that the engine's own implied order book never crosses, that a bid never sits above an ask. And level four is the deepest: we render the engine's entire sequence of state transitions as a mathematical graph, and compute the graph-edit-distance against the reference. If the two graphs diverge, the engine is non-deterministic — and we catch it. Four levels, every submission, no exceptions.

Segment 6 — Scoring & Anomaly Detection

■ 5:20 – 6:30

On screen — *Composite score formula; then the ML anomaly section / radar chart.*

Once an engine passes validation, UORA computes a single composite score. The formula is deliberately simple and deliberately unforgiving. The numerator rewards what we want: raw throughput, multiplied by correctness rate, multiplied by success rate. The denominator punishes what we do not want: tail latency, plus a squared penalty for wasted CPU and memory. Because correctness is a multiplier, an engine that is only half correct does not lose a few points — it loses half its entire score. And because the latency penalty is convex, doubling your worst-case latency quadruples the damage. But a clever contestant might try to cheat — hard-code responses to known test patterns, or fake a perfectly flat latency to look fast. So on top of scoring, UORA runs a machine-learning anomaly detector. An isolation forest, trained on the profile of a healthy engine, watches eight behavioral features at once — latency entropy, throughput variance, pattern correlation, state-transition distance, and more. An engine that looks statistically unlike anything healthy gets flagged automatically — and the judges see exactly why.

Segment 7 — Live Demonstration

6:30 – 7:45

On screen — *Dashboard: upload advanced_engine.py, watch pipeline, then the scored card.*

But none of this means anything if it does not actually run. So let us prove it. Here is the live console, signed in to a real account. I am going to drop in a real Python matching engine — two hundred lines that implement full price-time priority, all four order types, and self-trade prevention. Watch the pipeline. The platform detects the language, queues the build, compiles the engine, and deploys it into the sandbox — every stage lighting up green in real time. The build log on the right is not a scripted animation; that is the actual compiler output, streaming live. Now the bot fleet hits it. And there — a real score. A composite of two hundred and thirty, a hundred percent correctness, twenty-nine thousand orders per second, and a clean anomaly rating. Every one of those numbers came from a real process, accepting real HTTP traffic, producing real fills. And to prove the detector works — when we submit a deliberately broken engine, it is fast, but only eighteen percent correct, and the platform flags it instantly. Speed alone does not win.

Segment 8 — Close

7:45 – 8:30

On screen — *Leaderboard with the ranked entry; Reports tab → Download PDF; UORA logo.*

And the moment that score lands, the leaderboard updates for everyone, live, over a server-sent event stream. Open the reports tab, and the full performance audit is right there — tail latency, fill correctness, anomaly status — downloadable as a PDF that ships alongside the source. That is UORA. Compile, sandbox, replay, validate, score — the entire journey from a raw source file to a ranked, reproducible leaderboard entry, in under thirty seconds, with every number honest and every number repeatable. It is the testing platform that high-frequency trading has always needed, and never had. Thank you for watching. The complete source code is on GitHub, the platform is live on Google Cloud, and I would love for you to run it yourself. This has been UORA, submitted by Vansh Gupta, I I I T Bhopal, for I I C P C twenty twenty-six. Thank you.

End of narration · total ≈ 8:00